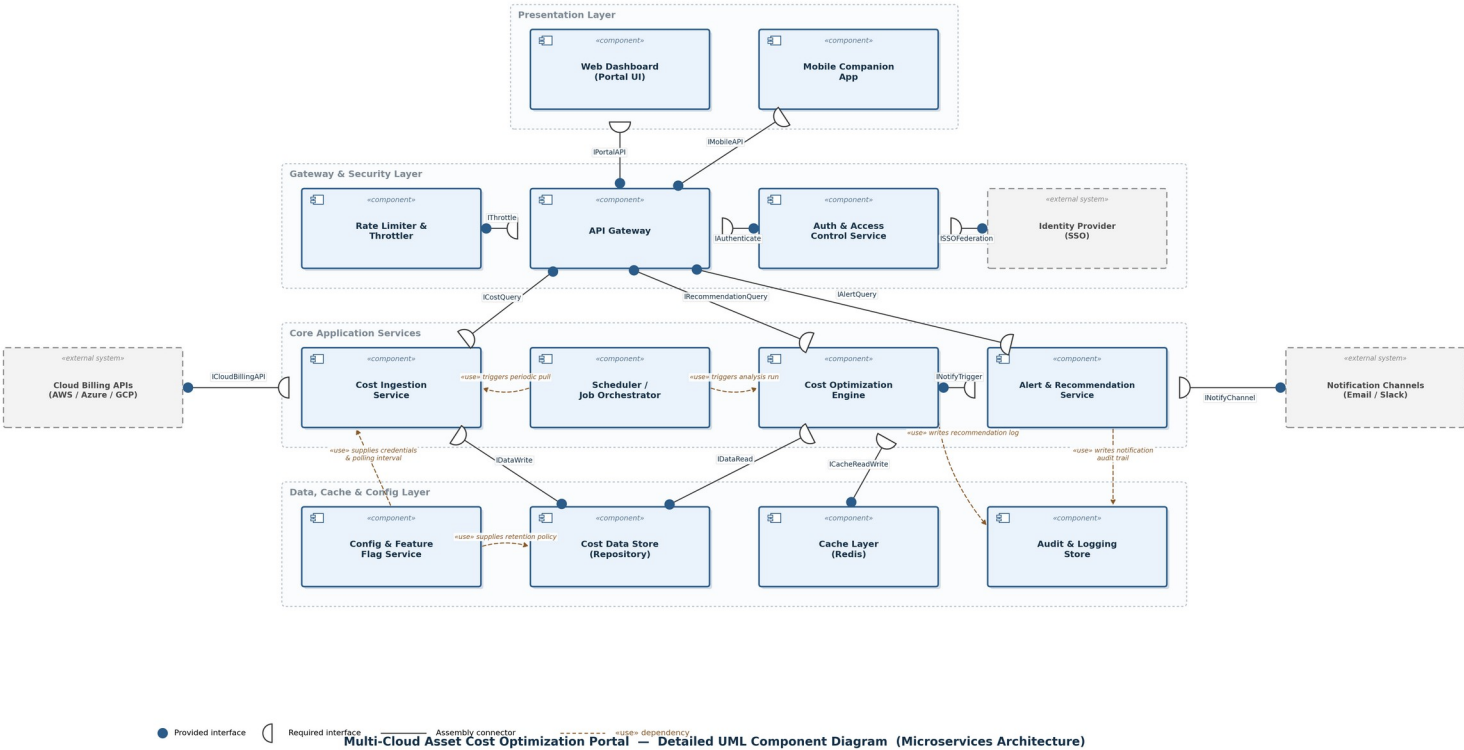




LAB 3 — Architecture Justification

Multi-Cloud Asset Cost Optimization Portal

Samarth A Ambekar | SRN: PES1UG24AM243 | Scenario 45 | Domain: Developer Tools & IT Operations | Actors: FinOps Analyst, Cloud Administrator

Architecture Selection: We went with a Microservices Architecture for the Multi-Cloud Asset Cost Optimization Portal — here's the thinking behind that.

Reason 1 — the two main jobs this system does just don't behave the same way

On one side, the portal is constantly pinging AWS, Azure and GCP to pull billing and usage numbers — that's a steady, lightweight job that never really stops. On the other side, there's the actual cost-optimization logic the FinOps Analyst depends on, like spotting idle resources or recommending a cheaper instance size, and that's a much heavier, bursty job (think end-of-month reviews). Cramming both into one layered or client-server system would mean they're forced to scale together even though they don't need to. Splitting them into a Cost Ingestion Service and a Cost Optimization Engine means we can scale, deploy and tweak each one on its own schedule instead.

Reason 2 — if one cloud provider has a bad day, the rest of the system shouldn't notice

AWS, Azure and GCP each run their own billing API with their own quirks, rate limits and failure modes. We kept all of that provider-specific handling inside the Cost Ingestion Service and nowhere else, so if one provider changes its API or just goes down for a bit, the damage stays right there — it doesn't ripple out and take the Cloud Administrator's dashboard or the optimization engine down with it. That's really the whole reason to break a system into services instead of shipping one big monolith.

Security advantage

Every login and permission check goes through a separate Auth & Access Control Service that sits in front of the API Gateway, which means the cloud provider keys the Ingestion Service is holding never get anywhere near the Web Dashboard or the person using it. It also makes it easy to draw the line between roles — FinOps Analysts can view and recommend, Cloud Administrators can actually approve and act — and since each service is deployed separately, patching one doesn't mean touching everything else.

Performance benefit

The Optimization Engine works off data the Ingestion Service has already saved, so it runs in the background instead of holding up a live API Gateway request — the dashboard stays snappy even while a big analysis job is churning away. And when things get busy, like around month-end, we can just spin up more Optimization Engine instances without having to over-provision the lighter, simpler services alongside it.